

04장 tkinter 실전 (tkinter Practice)

tkinter 실전 개요

tkinter의 기본 위젯과 레이아웃은 물론, 스레드 같은 심화 주제까지 익혔습니다. 이제 그 요소들을 한 데 모아 실제로 동작하는 프로그램을 만들어 볼 차례입니다. 이 장에서는 일상이나 업무에서 쓸 만한 몇 가지 데스크톱 애플리케이션을 직접 만들어보며, tkinter를 다루는 실력과 응용력을 함께 키워 나갑니다. 입문자를 위해 다음 두 가지 원칙을 기반으로 이 장을 구성했습니다.

- 최소 기능에서 시작해 점진적으로 확장하기

처음부터 모든 기능을 구현하기보다, 동작하는 가장 단순한 형태를 만들고 여기에 기능을 하나씩 덧붙입니다.

- 어려운 로직은 작은 예제로 먼저 익히기

완성된 코드를 '한 줄씩 살펴보기'로 설명하는 경우도 있지만, 까다로운 로직은 그에 앞서 동작하는 최소한의 코드로 따로 구현해 원리를 익힌 뒤 완성 프로그램에 통합합니다.

단순히 코드를 따라 치는 데 그치지 않고, 프로그램을 어떤 순서로 기획하고 완성해 가는지 그 흐름을 함께 짚어 보겠습니다.

뽀모도로 타이머 최소 기능

배운 개념들을 엮어 실전 프로젝트인 '뽀모도로 타이머'를 만들어보겠습니다. 이번 강좌는 아주 단순한 v0.1에서 시작해, 개발 중 마주하는 문제들을 하나씩 해결하며 v0.3까지 프로그램을 점진적으로 발전시켜 나갑니다.

v0.1 (기본 구현): 25분 카운트다운만 동작하는 가장 단순한 형태의 타이머를 만듭니다. v0.2 (구조 개선): 중복 실행 버그를 잡고 재시작 기능을 추가하며, 코드를 클래스(Class) 기반으로 깔끔하게 재구조화합니다. v0.3 (기능 확장): 타이머 시간 선택 기능을 더하고 UI 디자인(색상 등)을 개선해 실전 프로그램의 형태를 갖추니다.

하나의 작은 프로그램이 단계별로 디벨롭되는 과정을 따라가며, Tkinter 를 실전에서 어떻게 쓰이는지 확실하게 감을 잡아보세요.

- [시간이 줄어들게 만들기](#)
 - [목표](#)
 - [뽀모도로 v0.1](#)
 - [코드 한 줄씩 살펴보기](#)
- [클래스로 정리하고 리셋 기능 추가](#)
 - [목표](#)
 - [뽀모도로 v0.2](#)
 - [코드 한 줄씩 살펴보기](#)
- [시간을 고를 수 있게 하기](#)
 - [목표](#)
 - [뽀모도로 v0.3](#)
 - [코드 한 줄씩 살펴보기](#)

시간이 줄어들게 만들기

목표

- 시작 버튼을 누르면, 25:00에서 0까지 1초에 한 칸씩 줄어드는 화면을 보여주는 것입니다.
- 리셋도 없고, 시간 선택도 없고, 색도 없습니다.

뽀모도로 v0.1

```

import tkinter as tk

remaining = 25 * 60

def tick():
    global remaining
    m, s = divmod(remaining, 60)
    label.config(text=f"{m:02d}:{s:02d}")
    if remaining > 0:
        remaining -= 1
        root.after(1000, tick)

root = tk.Tk()
root.title("뽀 모 도 로 v0.1")
label = tk.Label(root, text="25:00", font=("Arial", 48))
label.pack(padx=40, pady=20)
tk.Button(root, text="시작", command=tick).pack(pady=10)
root.mainloop()

```

30줄도 안 되는 코드입니다.

코드 한 줄씩 살펴보기

- 남은 시간을 숫자 하나로 기억하기

```
remaining = 25 * 60
```

- 타이머의 핵심은 "지금 몇 초 남았는가"입니다. 25분은 $25 \times 60 = 1500$ 초이므로, `remaining` 이라는 상자에 `1500` 을 담아 둡니다.
- 여기서 굳이 `1500` 이라고 직접 쓰지 않고 `25 * 60` 이라고 쓴 이유가 있습니다. 코드를 읽는 사람이 "아, 25분짜리구나"라고 즉시 이해할 수 있기 때문입니다. **숫자에 의미를 담아 표현하는 습관**은 처음부터 들여 두면 좋습니다.

- 1초마다 실행되는 tick 함수

```
def tick():
    global remaining
```

- `tick` (똑딱)은 "1초가 흘렀다"는 사건을 처리하는 함수입니다.
- `global remaining` 는 함수 안에서 바깥에 있는 `remaining` 상자를 **고쳐 쓰겠다고 선언** 하는 부분입니다

- 나누기의 몫과 나머지를 한 번에 돌려주기

```
m, s = divmod(remaining, 60)
```

- `divmod(a, b)` 는 "a를 b로 나눈 몫과 나머지"를 돌려줍니다. 가령 `remaining` 이 1500 이면 `divmod(1500, 60)` 은 몫 25와 나머지 0을 줍니다.

- 문자열 표시 방법

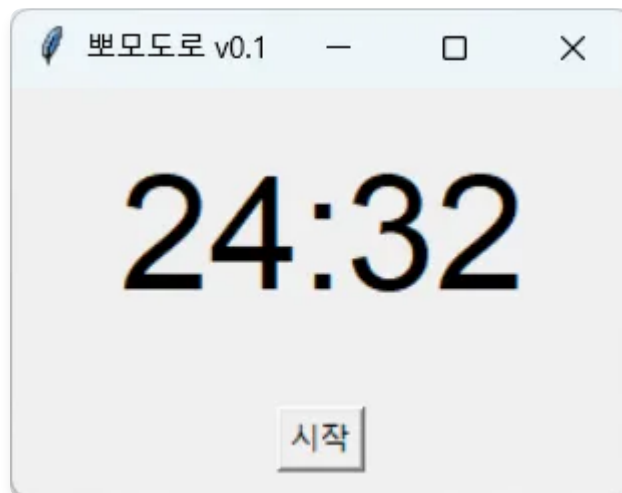
```
label.config(text=f"{m:02d}:{s:02d}")
```

- `f"{m:02d}:{s:02d}"` 가 낫설 수 있습니다. 이건 **f-문자열(f-string)** 이라는 표현으로, 중괄호 `{}` 안의 값을 문자열 안에 끼워 넣습니다. `:02d` 는 "정수(d)를 최소 두 자리로, 빈 자리는 0으로 채워라"는 뜻입니다.

- `remaining` 값 줄이기

```
if remaining > 0:
    remaining -= 1
    root.after(1000, tick)
```

- 아직 시간이 남았다면, 남은 초를 1 (`remaining -= 1`) 줄입니다.
- 1000 밀리초 뒤에 자기 자신인 `tick` 함수를 다시 실행하도록 예약 (`root.after`)합니다.



클래스로 정리하고 리셋 기능 추가

목표

- 흩어져 있던 코드를 클래스(class) 로 묶어 정리합니다.
- "리셋" 버튼을 추가해 언제든지 처음으로 되돌릴 수 있게 합니다.
- 시작 버튼을 여러 번 눌러도 타이머가 하나만 돌아가게 합니다.

뽀모도로 v0.2

```

import tkinter as tk

class PomodoroTimer:
    WORK_MIN = 25

    def __init__(self, root):
        self.root = root
        self.remaining = self.WORK_MIN * 60
        self.running = False

        self.label = tk.Label(root, text="25:00", font=("Arial", 48))
        self.label.pack(padx=40, pady=20)

        frame = tk.Frame(root)
        frame.pack(pady=10)
        tk.Button(frame, text="시작", command=self.start).pack(side="left", padx=5)
        tk.Button(frame, text="리셋", command=self.reset).pack(side="left", padx=5)

    def update_label(self):
        m, s = divmod(self.remaining, 60)
        self.label.config(text=f"{m:02d}:{s:02d}")

    def tick(self):
        if self.running and self.remaining > 0:
            self.remaining -= 1
            self.update_label()
            self.root.after(1000, self.tick)

    def start(self):
        if not self.running:
            self.running = True
            self.tick()

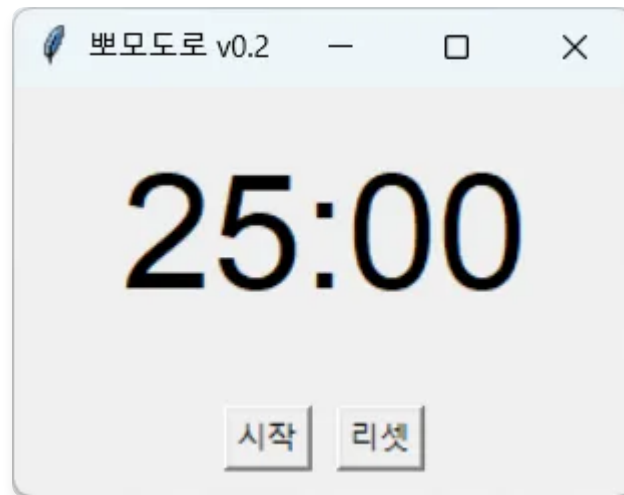
    def reset(self):
        self.running = False
        self.remaining = self.WORK_MIN * 60
        self.update_label()

root = tk.Tk()
root.title("뽕모도로 v0.2")

```

```
PomodoroTimer(root)
root.mainloop()
```

코드가 길어 보이지만, 하는 일은 v0.1과 거의 같습니다. 달라진 건 "정리 방식"과 "안전장치" 두 가지입니다.



코드 한 줄씩 살펴보기

- 관련된 값과 함수를 클래스로 묶기

```
class PomodoroTimer:
    ...
```

- V0.1에서는 remaining 값과 tick 함수가 따로 떨어져 있어, 값을 고치려면 global이라는 표시가 필요했습니다.
- 클래스는 서로 관련된 값과 함수를 하나의 상자에 담는 도구입니다. 남은 시간·실행 여부·라벨, 그리고 tick·start·reset 동작을 PomodoroTimer 상자 하나에 넣습니다.
- 상자 안의 값은 self. 를 붙여 꺼내 쓰므로, global이 더 이상 필요 없습니다.

- 설정값을 상수로 이름 붙이기


```
class PomodoroTimer:
    WORK_MIN = 25
```

- `WORK_MIN = 25`는 "작업 시간은 25분"이라는 설정값입니다. 대문자는 "잘 바뀌지 않는 값"이라는 관례적 신호입니다.
- 나중에 30분으로 바꾸고 싶으면 코드 곳곳을 뒤질 필요 없이 이 한 줄만 고치면 됩니다.

- 타이머가 생길 때 한 번 실행되는 **init**

```
def __init__(self, root):
    self.root = root
    self.remaining = self.WORK_MIN * 60
    self.running = False
```

- `__init__` (앞뒤 밑줄 두 개)은 맨 아래에서 `PomodoroTimer(root)` 로 객체를 만드는 순간 **자동으로 딱 한 번** 실행되는 초기 설정 함수입니다.
- 여기서 계속 나오는 `self` 는 "이 상자 자신"을 가리킵니다. `self.root` , `self.remaining` 처럼 상자 안에 값을 보관합니다.
- `self.running = False` 는 "지금 돌아가는 중인가?"를 나타내며, 이것이 겹침 버그를 푸는 열쇠입니다.

- 여러 번 눌러도 하나만 — start

```
def start(self):
    if not self.running:      # 이미 돌면 무시
        self.running = True
        self.tick()
```

- `if not self.running:` 은 "지금 돌아가는 중이 **아니라면**"이라는 뜻입니다. 이때만 `running` 을 True로 바꾸고 `tick` 을 시작합니다.
- 이미 돌아가는 중이면 `if` 를 건너뛰어 **아무 일도 일어나지 않습니다**. 그래서 시작을 열 번 눌러도 타이머는 딱 하나만 돌아갑니다.

- 처음으로 되돌리는 reset

```
def reset(self):
    self.running = False
    self.remaining = self.WORK_MIN * 60
    self.update_label()
```

- `running` 을 False로 바꾸면 `tick` 의 `if self.running ...` 조건이 거짓이 되어 다음 `after` 예약이 끊기고, 반복이 자연스럽게 멈춥니다.
- 남은 시간을 25분으로 되돌리고, 화면을 `25:00` 으로 새로 그립니다. 깃발(**running**) 하나로 흐름 전체를 제어하는 셈입니다.

시간을 고를 수 있게 하기

목표

- 20분 / 25분 / 30분 중에서 골라 시작할 수 있게 합니다.
- 배경에 색을 입혀 집중하기 좋은 화면으로 꾸밈니다.

뽀모도로 v0.3

```

import tkinter as tk

class PomodoroTimer:
    BG = "#e63946"

    def __init__(self, root):
        self.root = root
        self.running = False
        self.minutes = tk.IntVar(value=25)          # 선택된 분 (20/25/30)
        self.remaining = self.minutes.get() * 60
        root.config(bg=self.BG)

        self.label = tk.Label(root, text="25:00", font=("Arial", 60),
                               fg="white", bg=self.BG)
        self.label.pack(padx=40, pady=(0, 10))

        radios = tk.Frame(root, bg=self.BG)
        radios.pack()
        for m in (20, 25, 30):
            tk.Radiobutton(radios, text=f"{m}분", value=m, variable=self.minutes,
                           command=self.set_time, bg=self.BG, fg="white",
                           selectcolor=self.BG, activebackground=self.BG,
                           activeforeground="white").pack(side="left", padx=5)

        frame = tk.Frame(root, bg=self.BG)
        frame.pack(pady=20)
        tk.Button(frame, text="시작", command=self.start).pack(side="left", padx=5)
        tk.Button(frame, text="리셋", command=self.reset).pack(side="left", padx=5)

    def update_label(self):
        m, s = divmod(self.remaining, 60)
        self.label.config(text=f"{m:02d}:{s:02d}")

    def set_time(self):
        # 라디오 선택 시 호출
        self.running = False
        self.remaining = self.minutes.get() * 60
        self.update_label()

    def tick(self):
        if self.running and self.remaining > 0:

```

```

        self.remaining -= 1
        self.update_label()
        self.root.after(1000, self.tick)

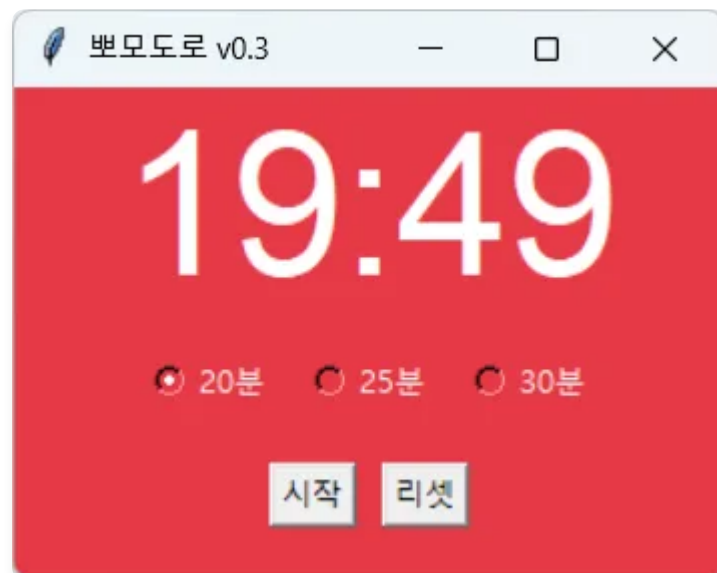
    def start(self):
        if not self.running:
            self.running = True
            self.tick()

    def reset(self):
        self.set_time()

root = tk.Tk()
root.title("뽀모도로 v0.3")
PomodoroTimer(root)
root.mainloop()

```

`tick` 과 `start` 는 V0.2와 동일합니다. 새로 배울 것은 `IntVar`, `Radiobutton`, `for` 반복으로 위젯 만들기입니다.



코드 한 줄씩 살펴보기

- 선택 상태를 담는 특별한 변수 `IntVar`

```
self.minutes = tk.IntVar(value=25)
self.remaining = self.minutes.get() * 60
```

- 라디오 버튼과 연결하려면 **tkinter 전용 변수인 IntVar** 가 필요합니다.
- 사용자가 "30분" 버튼을 고르면 이 변수 값이 자동으로 30이 됩니다. 반대로 값을 꺼낼 때는 `self.minutes.get()` 처럼 `.get()` 을 씁니다.
- 즉 **IntVar** 는 "화면의 선택 상태와 코드의 값을 이어 주는 다리"입니다.

- 계속 바꿀 라벨은 이름 붙여 보관하기

```
self.label = tk.Label(root, text="25:00", font=("Arial", 60),
                      fg="white", bg=self.BG)
self.label.pack(padx=40, pady=(0, 10))
```

타이머 라벨은 `update_label` 에서 계속 글자를 바꿔야 하므로, 나중에 찾을 수 있도록 `self.label` 에 이름을 붙여 보관합니다.

- for 반복으로 라디오 버튼 만들기

```
for m in (20, 25, 30):
    tk.Radiobutton(radios, text=f"{m}분", value=m, variable=self.minutes,
                  command=self.set_time, bg=self.BG, fg="white",
                  selectcolor=self.BG, activebackground=self.BG,
                  activeforeground="white").pack(side="left", padx=5)
```

- `Radiobutton` 은 여러 선택지 중 **하나만** 고르는 버튼입니다. 하나를 누르면 나머지는 저절로 풀립니다.
- 버튼이 세 개라 세 줄을 복사하는 대신 `for` 로 만듭니다. `(20, 25, 30)` 을 `m` 에 하나씩 넣어 "20분/25분/30분" 버튼을 찍어냅니다.
- `variable=self.minutes` 로 세 버튼을 같은 변수에 묶고, `value=m` 으로 고를 값을, `command=self.set_time` 으로 선택 시 실행할 함수를 정합니다.

- 선택이 바뀌면 시간을 다시 맞추는 `set_time`

```
def set_time(self):
    self.running = False
    self.remaining = self.minutes.get() * 60
    self.update_label()
```

- 라디오 버튼을 누르면 실행됩니다. 돌아가던 타이머를 멈추고, 방금 고른 분으로 남은 시간을 새로 계산하고, 화면을 갱신합니다.
- "30분"을 누르면 `self.minutes.get()` 이 30을 돌려주므로 `remaining` 은 1800초가 되고, 화면은 곧바로 `30:00` 으로 바뀝니다.

뽀모도로 타이머 완성

앞서 뽀모도로 타이머의 최소 기능으로 카운트다운을 만들고, 클래스로 정리하고, 시간을 선택하고 색을 입혔습니다. 그런데 시간은 오직 숫자로만 보였습니다.

이번에는 시간이 줄어드는 모습을 원형 그래프(파이) 로 그려 봅시다. 색칠된 부채꼴이 조금씩 줄어들고, 그 한가운데에 남은 시간이 적히는 형태입니다.

- [줄어드는 원형 그래프](#)
 - [목표](#)
 - [줄어드는 원형 그래프](#)
 - [코드 한 줄씩 살펴보기](#)
- [남은 시간을 원형 그래프로 그리기](#)
 - [목표](#)
 - [뽀모도로 v0.4](#)
 - [코드 한 줄씩 살펴보기](#)

줄어드는 원형 그래프

목표

- 뽀모도로 타이머에서 '시각적 효과 줄어드는 원형 그래프'를 그리는 로직이 가장 복잡할 수 있습니다.
- 이를 쉽게 이해하기 위해 줄어드는 원형 그래프 로직만 별도의 실행되는 코드로 작성해봅니다.

줄어드는 원형 그래프

- canvas 위젯을 사용해 60초 동안 원형 그래프가 줄어드는 코드입니다."

```

import tkinter as tk

COLOR = "#e63946"
total = 1 * 60
remaining = total

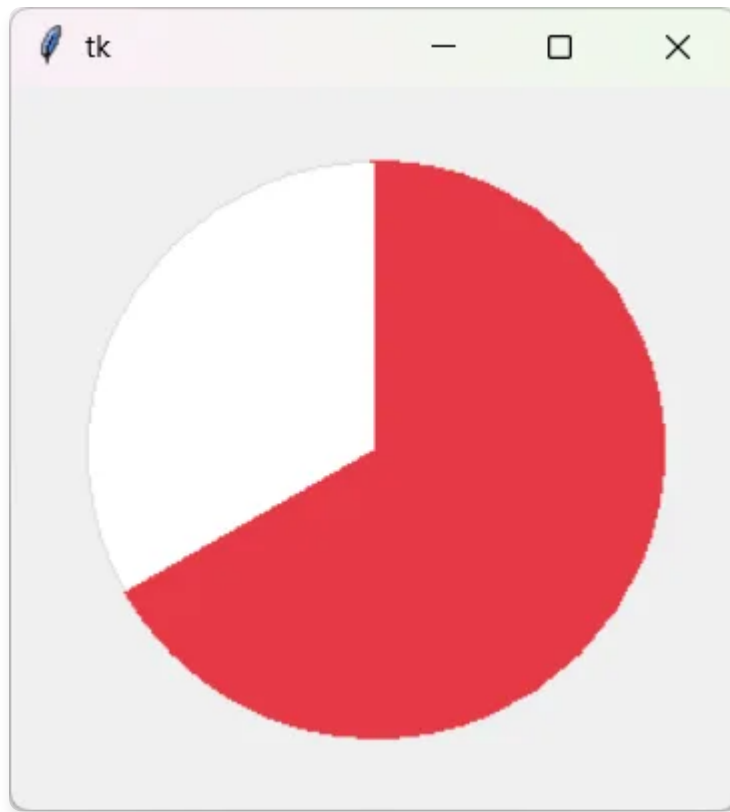
def draw():
    global remaining
    canvas.delete("all")
    box = (10, 10, 240, 240)
    canvas.create_oval(*box, fill="white", outline="#dddddd")

    frac = remaining / total
    if frac >= 1:
        canvas.create_oval(*box, fill=COLOR, outline="")
    elif frac > 0:
        canvas.create_arc(*box, start=90, extent=-360 * frac,
                          fill=COLOR, outline="")

    if remaining > 0:
        remaining -= 1
        root.after(1000, draw)

root = tk.Tk()
canvas = tk.Canvas(root, width=250, height=250, highlightthickness=0)
canvas.pack(padx=20, pady=20)
draw()
root.mainloop()

```

코드 한 줄씩 살펴보기

- 캔버스 모든 도형 지우기

```
canvas.delete("all")
```

애니메이션은 "지우고 → 다시 그리기"의 반복이라, 매번 깨끗한 상태에서 시작합니다.

- 사각형 영역 좌표

```
box = (10, 10, 240, 240)
```

box는 원을 그릴 사각형 영역의 좌표입니다. 좌측 상단 (10, 10)부터 우측 하단 (240, 240)까지입니다.

- 배경 원 그리기

```
canvas.create_oval(*box, fill="white", outline="#dddddd")
```

먼저 흰색 바탕 원을 그립니다. 나중에 붉은 파이가 줄어들어도 그 자리에 흰 원이 남아 있어서 "빈 부분"처럼 보이게 하는 배경입니다. `*box` 는 튜플을 풀어서 `create_oval(10, 10, 240, 240, ...)` 처럼 전달하는 문법입니다.

- 진행률 계산과 채우기

```
frac = remaining / total
```

`frac` 은 남은 비율입니다. 시작할 때 `remaining == total == 60`이라 `frac`은 1.0이고, 시간이 지날수록 0을 향해 줄어듭니다.

- 짝 찬 붉은 원 그리기

```
if frac >= 1:
    canvas.create_oval(*box, fill=COLOR, outline="")
```

비율이 1 이상, 즉 아직 하나도 안 줄었을 때는 짝 찬 붉은 원을 그립니다.

- 부채꼴 파이 조각 그리기

```
elif frac > 0:
    canvas.create_arc(*box, start=90, extent=-360 * frac,
                      fill=COLOR, outline="")
```

- 여기가 핵심입니다. `create_arc` 는 부채꼴(파이 조각)을 그립니다.
- `start=90` : 그리기 시작 각도가 90도, 즉 12시 방향(맨 위)입니다.
- `extent=-360 * frac` : 시작점에서 얼마만큼 더 그릴지의 각도입니다. 음수라서 시계 방향으로 그려집니다.

남은 시간을 원형 그래프로 그리기

목표

- 남은 시간을 원형 그래프(파이) 로 보여줍니다.
- 원 한가운데에 `mm:ss` 시간을 함께 표시합니다.

뽀모도로 v0.4

```

import tkinter as tk

class PomodoroTimer:
    COLOR = "#e63946"
    SIZE = 250 # 캔버스 한 변 길이(px)

    def __init__(self, root):
        self.root = root
        self.running = False
        self.minutes = tk.IntVar(value=25) # 선택된 분 (20/25/30)
        self.total = self.minutes.get() * 60
        self.remaining = self.total

        self.canvas = tk.Canvas(root, width=self.SIZE, height=self.SIZE,
                                highlightthickness=0)
        self.canvas.pack(pady=15)

        radios = tk.Frame(root)
        radios.pack()
        for m in (20, 25, 30):
            tk.Radiobutton(radios, text=f"{m}분", value=m, variable=self.minutes,
                           command=self.set_time).pack(side="left", padx=5)

        frame = tk.Frame(root)
        frame.pack(pady=(5, 15))
        tk.Button(frame, text="시작", command=self.start).pack(side="left", padx=5)
        tk.Button(frame, text="리셋", command=self.reset).pack(side="left", padx=5)

        self.draw()

    def set_time(self): # 라디오 선택 시 호출
        self.running = False
        self.total = self.minutes.get() * 60
        self.remaining = self.total
        self.draw()

    def draw(self):
        self.canvas.delete("all")
        pad = 10
        box = (pad, pad, self.SIZE - pad, self.SIZE - pad)

```

```

self.canvas.create_oval(*box, fill="white", outline="#dddddd")

frac = self.remaining / self.total          # 남은 비율 (1 → 0)
if frac >= 1:
    self.canvas.create_oval(*box, fill=self.COLOR, outline="")
elif frac > 0:
    # 12시 방향에서 시계방향으로
    self.canvas.create_arc(*box, start=90, extent=-360 * frac,
                           fill=self.COLOR, outline="")

m, s = divmod(self.remaining, 60)
self.canvas.create_text(self.SIZE // 2, self.SIZE // 2,
                        text=f"{m:02d}:{s:02d}", font=("Arial", 36))

def tick(self):
    if self.running and self.remaining > 0:
        self.remaining -= 1
        self.draw()
        self.root.after(1000, self.tick)

def start(self):
    if not self.running:
        self.running = True
        self.tick()

def reset(self):
    self.set_time()

root = tk.Tk()
root.title("뽕모도로 v0.4")
PomodoroTimer(root)
root.mainloop()

```

`tick` · `start` · `reset` · `set_time` 의 뼈대는 V0.3과 같습니다.



코드 한 줄씩 살펴보기

- 글자 대신 그림을 그리는 공간, Canvas

```
self.canvas = tk.Canvas(root, width=self.SIZE, height=self.SIZE,
                        highlightthickness=0)
self.canvas.pack(pady=15)
```

- `Canvas` (캔버스) 위젯을 씁니다. 원·선·글자를 자유롭게 그릴 수 있습니다.
- `width` · `height` 로 250×250 정사각형 그림판을 만듭니다.
- `highlightthickness=0` 은 캔버스가 기본으로 두르는 얇은 테두리를 없애 깔끔하게 만듭니다.

- 두 개의 시간, total과 remaining

```
self.total = self.minutes.get() * 60
self.remaining = self.total
```

- v0.3에는 `remaining` 하나뿐이었습니다. 그런데 원을 "얼마나 채울지" 계산하려면 **전체**가 몇 초였는지도 알아야 합니다.
- `total` 은 처음 정한 전체 시간(변하지 않음), `remaining` 은 줄어드는 남은 시간입니다. 두 값의 비율이 곧 원을 채우는 정도가 됩니다.

- 원 한가운데에 시간 쓰기

```
m, s = divmod(self.remaining, 60)
self.canvas.create_text(self.SIZE // 2, self.SIZE // 2,
                        text=f"{m:02d}:{s:02d}", font=("Arial", 36))
```

`create_text` 는 지정한 좌표를 **중심**으로 글자를 놓습니다. `SIZE // 2` 는 $250 \div 2 = 125$, 곧 캔버스 한가운데입니다.

JSON 연락처 관리

JSON 파일을 다루는 법을 간략히 살펴보고 JSON 파일 기반의 tkinter 연락처 앱을 만들어 보겠습니다.

- [JSON 파일 이해하기](#)
 - [JSON이란 무엇인가?](#)
 - [JSON의 생김새](#)
 - [파이썬에서 JSON 파일 읽고 쓰기](#)
 - [JSON 저장/불러오기 코드](#)
 - [코드 한 줄씩 살펴보기](#)
- [입력폼 생성하고 연락처 추가하기](#)
 - [목표](#)
 - [연락처 v0.1](#)
- [Treeview 위젯으로 연락처 보기](#)
 - [목표](#)
 - [연락처 v0.2](#)
 - [코드 한 줄씩 살펴보기](#)

JSON 파일 이해하기

JSON이란 무엇인가?

- JSON(JavaScript Object Notation)은 데이터를 텍스트로 표현하는 형식입니다.
- 읽기 쉽고 구조도 단순해서, 프로그램끼리 데이터를 주고받거나 파일에 데이터를 저장할 때 널리 쓰입니다.

JSON의 생김새

- 연락처 하나를 JSON으로 표현하면 아래와 같습니다.


```
{
  "name": "홍길동",
  "company": "한빛상사",
  "phone": "010-1234-5678"
}
```

- 중괄호 `{ }` 로 묶인 이 덩어리를 객체라고 부릅니다. 안에는 `"이름": 값` 형태의 쌍이 들어 있습니다.
- 왼쪽의 `"name"` 은 항목의 이름(키), 오른쪽은 그 값입니다. 키는 반드시 큰따옴표로 감싸고, 쌍과 쌍 사이는 쉼표로 구분합니다.
- 연락처는 여러 개가 필요하므로, 객체들을 대괄호 `[ ]` 로 묶어 배열(목록)로 만듭니다.

파이썬에서 JSON 파일 읽고 쓰기

- 파이썬에는 JSON을 다루는 `json` 모듈이 기본으로 들어 있어 따로 설치할 필요가 없습니다. 핵심 함수는 두 개입니다.
- `json.load(f)` 는 파일에서 JSON을 읽어 파이썬 자료형으로 바꿉니다. 파일 속 배열은 리스트로, 객체는 딕셔너리로 변환되어 돌아옵니다.
- `json.dump(데이터, f)` 는 반대로 파이썬 자료형을 JSON 텍스트로 바꿔 파일에 씁니다.

JSON 저장/불러오기 코드

- 앱에서 사용할 저장/불러오기 코드를 보면 이 두 함수가 어떻게 쓰이는지 알 수 있습니다.

```
import os, json

FILE = "contacts.json"

def load():
    if os.path.exists(FILE):
        with open(FILE, encoding="utf-8") as f:
            return json.load(f)
    return []

def save():
    with open(FILE, "w", encoding="utf-8") as f:
        json.dump(contacts, f, ensure_ascii=False, indent=2)

contacts = load()
save()
```

실행하면 코드와 같은 경로에 'contacts.json' 파일이 생성됩니다.

코드 한 줄씩 살펴보기

- `load` 함수: 파일이 있는지 확인 후 읽기

```
if os.path.exists(FILE):
    with open(FILE, encoding="utf-8") as f:
        return json.load(f)
return []
```

- `os.path.exists` 로 파일이 있는지 먼저 확인합니다.
- 있으면 열어서 `json.load` 로 내용을 읽어 리스트로 돌려줍니다.
- 없으면(처음 실행이라 파일이 아직 만들어지지 않았으면) 빈 리스트 `[]` 를 돌려줍니다. 이렇게 해 두면 파일이 없어도 오류 없이 시작할 수 있습니다.

- `save` 함수: 쓰기 모드로 열어 리스트를 기록

```
with open(FILE, "w", encoding="utf-8") as f:
    json.dump(contacts, f, ensure_ascii=False, indent=2)
```

- `open` 에 `"w"` 를 넘겨 쓰기 모드로 파일을 열고, `json.dump` 로 `contacts` 리스트를 파일에 기록합니다.
- `ensure_ascii=False` 는 한글이 알아보기 힘든 코드로 저장되지 않고 한글 그대로 저장되게 합니다.
- `indent=2` 는 저장된 파일에 들여쓰기를 넣어, 사람이 직접 열어 봤을 때 앞에서 본 예시처럼 보기 좋게 정렬해 줍니다.

■ 알아두기

`with open(...)` 구문과 인코딩 지정

`with open(...)` 구문을 쓰면 작업이 끝났을 때 파일이 자동으로 닫혀 안전합니다.
`encoding="utf-8"` 은 한글이 깨지지 않도록 문자 인코딩을 지정하는 부분입니다.

입력폼 생성하고 연락처 추가하기

목표

- `contacts.json` 파일에 '이름','회사','연락처'를 추가할 수 있는 최소한의 GUI 폼을 작성합니다.

연락처 v0.1

```

import tkinter as tk
import os, json

FILE = "contacts.json"

def load():
    if os.path.exists(FILE):
        with open(FILE, encoding="utf-8") as f:
            return json.load(f)
    return []

def save():
    with open(FILE, "w", encoding="utf-8") as f:
        json.dump(contacts, f, ensure_ascii=False, indent=2)

def add():
    contacts.append({
        "name": e_name.get(),
        "company": e_company.get(),
        "phone": e_phone.get(),
    })
    save()

contacts = load()

root = tk.Tk()
root.title("연락처 v0.1")

tk.Label(root, text="이름").grid(row=0, column=0)
tk.Label(root, text="회사").grid(row=0, column=1)
tk.Label(root, text="전화").grid(row=0, column=2)

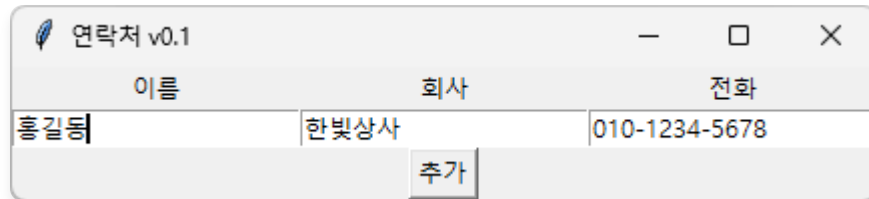
e_name = tk.Entry(root)
e_company = tk.Entry(root)
e_phone = tk.Entry(root)
e_name.grid(row=1, column=0)
e_company.grid(row=1, column=1)
e_phone.grid(row=1, column=2)

tk.Button(root, text="추가", command=add).grid(row=3, column=1)

```

```
root.mainloop()
```

추가만 가능하고 저장된 값은 JSON 파일을 직접 열어 확인해야 하는, 아직 다듬을 부분이 많은 코드입니다.



Treeview 위젯으로 연락처 보기

목표

- `ttk.Treeview` 위젯으로 'contacts.json' 파일 내 연락처 정보를 엑셀 시트처럼 출력합니다.
- 연락처 삭제 기능을 추가합니다.

연락처 v0.2

```

import tkinter as tk
from tkinter import ttk, messagebox
import os, json

FILE = "contacts.json"

def load():
    if os.path.exists(FILE):
        with open(FILE, encoding="utf-8") as f:
            return json.load(f)
    return []

def save():
    with open(FILE, "w", encoding="utf-8") as f:
        json.dump(contacts, f, ensure_ascii=False, indent=2)

def refresh():
    tree.delete(*tree.get_children())
    for c in contacts:
        tree.insert("", "end", values=(c["name"], c["company"], c["phone"]))

def add():
    name, company, phone = e_name.get(), e_company.get(), e_phone.get()
    if not name:
        messagebox.showwarning("확인", "이름을 입력하세요.")
        return
    contacts.append({"name": name, "company": company, "phone": phone})
    save()
    refresh()
    for e in (e_name, e_company, e_phone):
        e.delete(0, "end")

def delete():
    sel = tree.selection()
    if not sel:
        return
    idx = tree.index(sel[0])
    contacts.pop(idx)
    save()
    refresh()

```

```

contacts = load()

root = tk.Tk()
root.title("연락처 v0.2")

input_frame = tk.LabelFrame(root, text="입력", padx=10, pady=10)
input_frame.pack(padx=10, pady=10, fill="x")

tk.Label(input_frame, text="이름").grid(row=0, column=0)
tk.Label(input_frame, text="회사").grid(row=0, column=1)
tk.Label(input_frame, text="전화").grid(row=0, column=2)

e_name = tk.Entry(input_frame)
e_company = tk.Entry(input_frame)
e_phone = tk.Entry(input_frame)
e_name.grid(row=1, column=0, padx=2)
e_company.grid(row=1, column=1, padx=2)
e_phone.grid(row=1, column=2, padx=2)

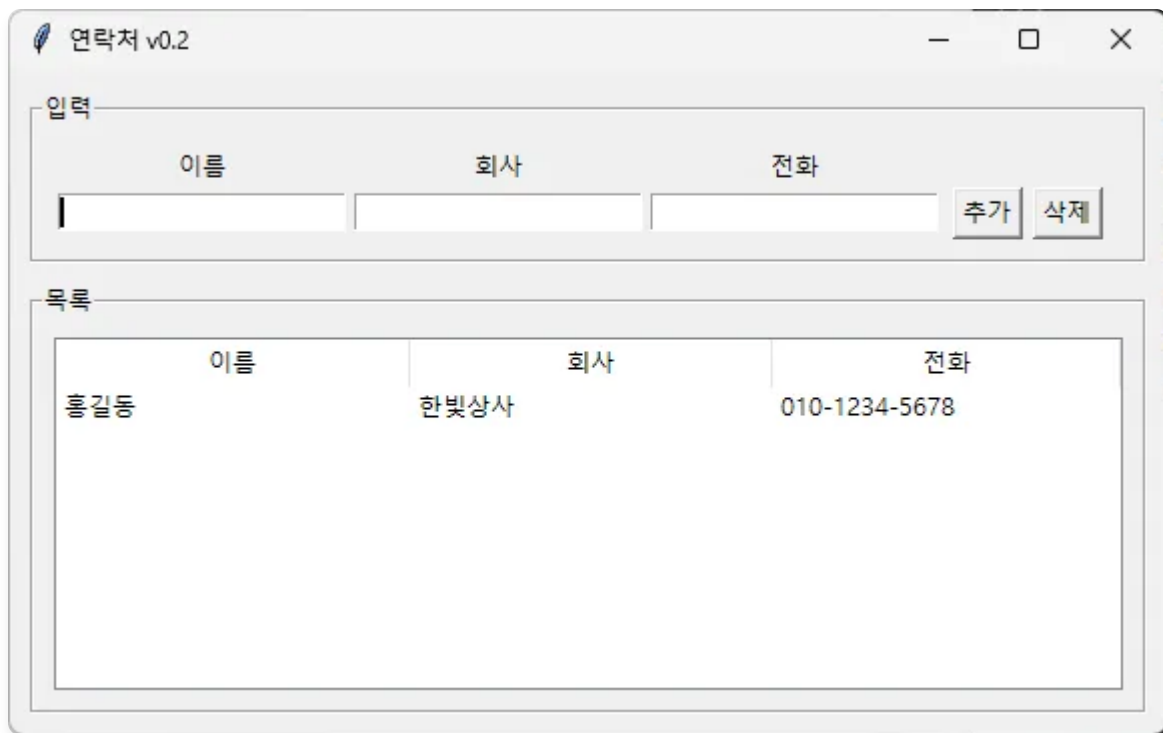
tk.Button(input_frame, text="추가", command=add).grid(row=1, column=3, padx=5)
tk.Button(input_frame, text="삭제", command=delete).grid(row=1, column=4)

list_frame = tk.LabelFrame(root, text="목록", padx=10, pady=10)
list_frame.pack(padx=10, pady=(0, 10), fill="both")

tree = ttk.Treeview(list_frame, columns=("name", "company", "phone"), show="headings", height=10)
tree.heading("name", text="이름")
tree.heading("company", text="회사")
tree.heading("phone", text="전화")
tree.pack()

refresh()
root.mainloop()

```



코드 한 줄씩 살펴보기

- `refresh` 함수: 표를 비우고 다시 채우기

```
tree.delete(*tree.get_children())
for c in contacts:
    tree.insert("", "end", values=(c["name"], c["company"], c["phone"]))
```

- `tree.get_children()` 은 표에 들어 있는 모든 행을 가져오고, `delete(*...)` 로 그것들을 한 번에 지웁니다. 즉 표를 깨끗이 비웁니다.
- 그런 다음 `contacts` 리스트를 하나씩 돌면서 이름·회사·전화를 한 행씩 다시 넣습니다.

- `add` 함수: 리스트에 넣고 저장·갱신·입력칸 비우기

```
contacts.append({"name": name, "company": company, "phone": phone})
save()
refresh()
for e in (e_name, e_company, e_phone):
    e.delete(0, "end")
```


- 읽어 온 값을 딕셔너리로 묶어 `contacts` 리스트에 추가합니다.
- 곧바로 `save()` 로 파일에 저장하고, `refresh()` 로 표를 새로 그립니다. 이 순서를 지켜야 데이터·파일·화면이 모두 일치합니다.
- 마지막으로 세 입력칸을 `delete(0, "end")` 로 비워, 다음 입력을 바로 할 수 있게 합니다.

- `delete` 함수: 선택한 행을 찾아 지우기

```
sel = tree.selection()
if not sel:
    return
idx = tree.index(sel[0])
contacts.pop(idx)
save()
refresh()
```

- `tree.selection()` 은 사용자가 표에서 클릭해 고른 행을 돌려줍니다.
- `tree.index(sel[0])` 으로 선택된 행이 몇 번째인지 위치(`idx`)를 구하고, 같은 위치의 항목을 `contacts.pop(idx)` 으로 리스트에서 제거합니다.
- 표의 순서와 리스트의 순서가 같기 때문에, 화면에서 고른 위치가 곧 리스트의 위치가 됩니다.

- 목록 표(Treeview) 만들기

```
tree = ttk.Treeview(list_frame, columns=("name", "company", "phone"), show="headings", height=10)
```

- `Treeview` 는 여러 줄·여러 열로 된 표를 보여 주는 위젯입니다. `columns` 로 세 개의 열을 정의합니다.

MVP 패턴 연락처 앱

절차적 코드로 작성되었던 연락처 앱을 OOP 기반의 MVP 패턴으로 수정하겠습니다. 지금 규모에선 큰 이점이 없지만, 기능이 늘어날수록 코드 관리가 수월해집니다"

- [모델](#)
 - [설명](#)
 - [model.py](#)
- [뷰](#)
 - [설명](#)
 - [view.py](#)
- [프리젠터](#)
 - [설명](#)
 - [presenter.py](#)
 - [main.py](#)

모델

설명

- 데이터를 담당하는 모델부터 만듭니다.
- 절차적 코드에서 흩어져 있던 load, save, 그리고 연락처 리스트를 다루던 부분이 하나의 클래스로 모입니다.
- 모델은 데이터만 다루고 화면이나 입력은 전혀 알지 못한다는 점이 핵심입니다

model.py

```

import os, json

class ContactModel:
    def __init__(self, file_path="contacts.json"):
        self.file_path = file_path
        self.contacts = self._load()

    def _load(self):
        if os.path.exists(self.file_path):
            with open(self.file_path, encoding="utf-8") as f:
                return json.load(f)
        return []

    def _save(self):
        with open(self.file_path, "w", encoding="utf-8") as f:
            json.dump(self.contacts, f, ensure_ascii=False, indent=2)

    def get_all(self):
        return self.contacts

    def add(self, name, company, phone):
        self.contacts.append({"name": name, "company": company, "phone": phone})
        self._save()

    def delete(self, idx):
        self.contacts.pop(idx)
        self._save()

```

- 이름 앞에 밑줄이 붙은 `_load` 와 `_save` 는 클래스 내부에서만 쓰는 메서드라는 신호입니다.
- 바깥에서는 `get_all` , `add` , `delete` 처럼 밑줄 없는 메서드만 호출하고, 파일을 실제로 읽고 쓰는 세부 동작은 클래스 안에 감춰 둔 것입니다.
- `add` 나 `delete` 가 끝날 때마다 스스로 `_save` 를 호출하므로, 호출하는 쪽은 저장을 신경 쓸 필요가 없습니다

부

설명

- 화면을 담당하는 뷰입니다. 입력창·버튼·목록을 그리고 입력값을 넘겨주거나 목록을 다시 그립니다.
- 저장이나 검증은 하지 않고, 버튼을 눌렀을 때의 처리는 프리젠테어에게 맡깁니다

view.py

```

import tkinter as tk
from tkinter import ttk, messagebox

class ContactView:
    def __init__(self, root):
        self.root = root
        self.presenter = None
        self._setup_ui()

    def set_presenter(self, presenter):
        self.presenter = presenter

    def _setup_ui(self):
        input_frame = tk.LabelFrame(self.root, text="입력", padx=10, pady=10)
        input_frame.pack(padx=10, pady=10, fill="x")

        tk.Label(input_frame, text="이름").grid(row=0, column=0)
        tk.Label(input_frame, text="회사").grid(row=0, column=1)
        tk.Label(input_frame, text="전화").grid(row=0, column=2)

        self.e_name = tk.Entry(input_frame)
        self.e_company = tk.Entry(input_frame)
        self.e_phone = tk.Entry(input_frame)

        self.e_name.grid(row=1, column=0, padx=2)
        self.e_company.grid(row=1, column=1, padx=2)
        self.e_phone.grid(row=1, column=2, padx=2)

        tk.Button(input_frame, text="추가",
                  command=lambda: self.presenter.add()).grid(row=1, column=3, padx=5)
        tk.Button(input_frame, text="삭제",
                  command=lambda: self.presenter.delete()).grid(row=1, column=4)

        list_frame = tk.LabelFrame(self.root, text="목록", padx=10, pady=10)
        list_frame.pack(padx=10, pady=(0, 10), fill="both", expand=True)

        self.tree = ttk.Treeview(list_frame, columns=("name", "company", "phone"),
                                 show="headings", height=8)
        self.tree.heading("name", text="이름")

```

```

self.tree.heading("company", text="회사")
self.tree.heading("phone", text="전화")
self.tree.pack(fill="both", expand=True)

def get_inputs(self):
    return self.e_name.get(), self.e_company.get(), self.e_phone.get()

def clear_inputs(self):
    for e in (self.e_name, self.e_company, self.e_phone):
        e.delete(0, "end")

def get_selected_index(self):
    sel = self.tree.selection()
    if not sel:
        return None
    return self.tree.index(sel[0])

def render(self, contacts):
    self.tree.delete(*self.tree.get_children())
    for c in contacts:
        self.tree.insert("", "end", values=(c["name"], c["company"], c["phone"]))

def show_warning(self, title, message):
    messagebox.showwarning(title, message)

```

- 버튼의 `command` 가 `lambda: self.presenter.add()` 로 되어 있는 데 주목합니다.
- 이전에는 버튼이 직접 `add` 함수를 불렀지만, 이제는 프리젠테에게 "추가해 달라"고 요청만 합니다.
- `get_inputs`, `clear_inputs`, `render` 처럼 뷰가 제공하는 메서드들은 모두 화면을 읽거나 다시 그리는 창구 역할을 하며, 프리젠테는 이 창구를 통해서만 화면과 소통합니다.
- `view.py` 코드 하단에 아래 코드를 추가하면 UI만 확인할 수 있습니다.

```

if __name__ == "__main__":
    root = tk.Tk()
    view = ContactView(root)
    root.mainloop()

```

프리젠퍼

설명

- 서로 모르는 모델과 뷰 사이를 오가며 흐름을 조율하는 것이 프리젠퍼입니다
- 절차적 코드에서 `add`, `delete`, `refresh` 함수가 하던 "판단과 조율"의 로직이 이곳으로 모입니다

presenter.py

```
class ContactPresenter:
    def __init__(self, view, model):
        self.view = view
        self.model = model
        self.view.set_presenter(self)
        self.refresh()

    def refresh(self):
        self.view.render(self.model.get_all())

    def add(self):
        name, company, phone = self.view.get_inputs()
        if not name:
            self.view.show_warning("확인", "이름을 입력하세요.")
            return
        self.model.add(name, company, phone)
        self.refresh()
        self.view.clear_inputs()

    def delete(self):
        idx = self.view.get_selected_index()
        if idx is None:
            return
        self.model.delete(idx)
        self.refresh()
```

- `add` 의 흐름을 따라가 보면 세 부분이 어떻게 협력하는지 잘 드러납니다.

- 프리젠퍼는 먼저 뷰에서 입력값을 받아 오고(`get_inputs`), 이름이 비었는지 검증한 뒤, 문제가 없으면 모델에 저장을 지시하고(`model.add`), 마지막으로 화면을 다시 그리고 입력창을 비웁니다.
- 이름 검증 같은 "규칙"은 뷰도 모델도 아닌 프리젠퍼가 쥐고 있습니다.
- 생성자에서 `set_presenter` 로 자신을 뷰에 등록하고 곧바로 `refresh` 를 호출해, 앱이 켜지자마자 저장돼 있던 연락처가 화면에 나타나게 합니다.

main.py

- 세 파일을 모두 만들었으니, 이제 이들을 실제로 조립해 실행하는 진입점만 남았습니다.

```
import tkinter as tk

from model import ContactModel
from view import ContactView
from presenter import ContactPresenter

def main():
    root = tk.Tk()
    root.title("연락처 v0.3")

    view = ContactView(root)
    model = ContactModel()
    ContactPresenter(view, model)

    root.mainloop()

if __name__ == "__main__":
    main()
```

- main 함수가 하는 일은 세 조각을 만들어 연결하는 것뿐입니다.
- 뷰와 모델을 각각 생성한 뒤, 이 둘을 프리젠퍼에 넘겨 주면 나머지 연결은 프리젠퍼가 알아서 처리합니다.
- 조립 과정이 단순하다는 것 자체가 MVP 패턴이 역할을 잘 나눠 놓았다는 증거입니다.

연락처 v0.3

입력

이름	회사	전화

추가 삭제

목록

이름	회사	전화
홍길동	한빛상사	010-1234-5678
김철수	태화무역	010-9876-5432

네트워크 점검 스크립트

파이썬으로 **HTTP / ICMP / TCP** 세 가지 점검 기능을 갖춘 GUI 프로그램을 단계적으로 만들어 보는 시리즈입니다. 최종적으로는 tkinter를 이용해 여러 대상을 점검하고 결과를 보여 주는 GUI 앱을 완성하게 됩니다.

하지만 처음부터 GUI를 붙이면 "화면 코드"와 "실제로 동작하는 코드"가 뒤섞여 이해하기 어렵습니다. 그래서 먼저 **GUI 없이 점검 로직만** 만들어 보겠습니다.

- [점검 방식 세 가지](#)
 - [HTTP 점검](#)
 - [ICMP 점검 \(ping\)](#)
 - [TCP 점검](#)
- [점검 코드 작성](#)
 - [공통 부모 클래스 Checker](#)
 - [Http 점검 클래스 HttpChecker](#)
 - [ICMP 점검 클래스 IcmpChecker](#)
 - [TCP 점검 클래스 TcpChecker](#)
- [실행해 보기 \(REPL\)](#)
 - [HTTP 점검 실행](#)
 - [ICMP 점검 실행](#)
 - [TCP 점검 실행](#)

점검 방식 세 가지

- 네트워크에서 "서버가 살아 있는가?"를 확인하는 방법은 확인하려는 계층과 목적에 따라 방식이 달라집니다.
- 이 프로그램에서는 다음 세 가지를 사용합니다.

HTTP 점검

- 웹 서버가 정상적으로 페이지를 응답하는지 `urllib` 를 사용해 확인합니다.

- 대상 주소로 HTTP 요청을 보낸 뒤, 돌아오는 **상태 코드(status code)** 를 봅니다. `200` 이면 정상 응답이라는 뜻입니다.

ICMP 점검 (ping)

- 우리가 흔히 쓰는 `ping` 명령과 같습니다. `subprocess` 를 사용해 대상 호스트에 ICMP 패킷을 보내고 응답이 오는지 확인합니다.
- 호스트가 네트워크에 연결되어 있고 살아 있는지를 가장 가볍게 확인하는 방법입니다.

TCP 점검

- `socket` 을 사용해 특정 **포트**가 열려 있는지 확인합니다.
- 예를 들어 데이터베이스가 '3306', SSH가 '22' 포트에서 동작하는데, 그 포트로 실제 TCP 연결이 맺어지는지 시험합니다.

점검 코드 작성

- 이제 GUI 없이 점검만 수행하는 아래 코드를 `checker.py` 라는 이름으로 저장하세요.

```

import socket
import subprocess
import platform
import urllib.request

class Checker:
    def __init__(self, target):
        self.target = target

    def run(self):
        raise NotImplementedError # 하위 클래스에서 구현

class HttpChecker(Checker):
    def run(self):
        url = self.target
        if not url.startswith(("http://", "https://")):
            url = "http://" + url
        try:
            with urllib.request.urlopen(url, timeout=5) as resp:
                code = resp.getcode()
                return code == 200, f"상태 코드 {code}"
        except Exception as e:
            return False, f"실패: {e}"

class IcmpChecker(Checker):
    def run(self):
        count = "-n" if platform.system() == "Windows" else "-c"
        cmd = ["ping", count, "1", self.target]
        try:
            result = subprocess.run(cmd, capture_output=True, timeout=5)
            ok = result.returncode == 0
            return ok, "응답 성공" if ok else "응답 없음"
        except Exception as e:
            return False, f"실패: {e}"

class TcpChecker(Checker):

```

```
def __init__(self, target, port):
    super().__init__(target)
    self.port = int(port)

def run(self):
    try:
        with socket.create_connection((self.target, self.port), timeout=5):
            return True, "포트 열림"
    except Exception as e:
        return False, f"포트 닫힘: {e}"
```

공통 부모 클래스 Checker

- 세부 동작은 다르지만 "대상을 점검 후 결과를 반환한다"는 공통 골격을 정의합니다. 그래서 공통 부모 클래스 `Checker` 를 두었습니다.
- 공통 틀을 잡아 두면, GUI등 외부에서는 어떤 점검이든 `checker.run()` 한 줄로 일관되게 호출할 수 있습니다.
- 점검 대상 저장하기

```
def __init__(self, target):
```

점검할 대상을 `self.target` 에 저장합니다.

- 하위 클래스가 구현할 자리 만들기

```
def run(self):
    raise NotImplementedError # 하위 클래스에서 구현
```

`run(self)` : `raise NotImplementedError` 로 비워 둡니다. 이는 "이 메서드는 반드시 하위 클래스가 직접 구현해야 한다"는 약속입니다.

Http 점검 클래스 HttpChecker

- `urllib.request.urlopen` 으로 대상 URL에 요청을 보냅니다.
- 스킴 자동 보정하기

```
if not url.startswith(("http://", "https://")):
    url = "http://" + url
```

사용자가 `http://` 없이 `example.com` 만 입력해도 동작하도록, 스킴이 없으면 앞에 `http://` 를 붙입니다.

- 타임아웃으로 멈춤 방지하기

```
with urllib.request.urlopen(url, timeout=5) as resp:
```

`timeout=5` 로 5초 안에 응답이 없으면 실패로 처리합니다. 응답 없는 대상에서 프로그램이 멈추는 것을 방지합니다.

- 성공 판정하기

```
code = resp.getcode()
return code == 200, f"상태 코드 {code}"
```

상태 코드가 `200` 이면 성공으로 봅니다.

- 예외 잡아내기

```
except Exception as e:
    return False, f"실패: {e}"
```

연결 실패, 타임아웃 등 모든 예외를 잡아 실패 메시지로 돌려줍니다.

ICMP 점검 클래스 IcmpChecker

- 운영체제의 `ping` 명령을 `subprocess.run` 으로 실행합니다.
- OS별 옵션 판별하기

```
count = "-n" if platform.system() == "Windows" else "-c"
```

패킷을 몇 개 보낼지 지정하는 옵션이 OS마다 달라서, Windows면 `-n`, 그 외(Linux/macOS)면 `-c` 를 사용합니다. `platform.system()` 으로 현재 OS를 판별합니다.

- 패킷 1개만 보내기

```
cmd = ["ping", count, "1", self.target]
```

`"1"` 은 패킷을 1개만 보내라는 뜻입니다.

- 출력 붙잡아 두기

```
result = subprocess.run(cmd, capture_output=True, timeout=5)
```

`capture_output=True` 로 명령의 출력을 화면에 그대로 쏟아내지 않고 붙잡아 둡니다.

- 종료 코드로 판정하기

```
ok = result.returncode == 0
```

`ping` 의 종료 코드(`returncode`)가 `0` 이면 응답 성공으로 판단합니다.

TCP 점검 클래스 TcpChecker

- `socket.create_connection` 으로 대상의 특정 포트에 TCP 연결을 시도합니다.
- 부모 초기화 후 포트 저장하기

```
def __init__(self, target, port):
    super().__init__(target)
    self.port = int(port)
```

다른 두 클래스와 달리 `port` 가 추가로 필요하므로 `__init__` 을 재정의합니다.
`super().__init__(target)` 으로 부모의 초기화를 먼저 호출한 뒤, 포트를 정수로 변환해 저장합니다.

- 연결 성공으로 판정하기

```
with socket.create_connection((self.target, self.port), timeout=5):
    return True, "포트 열림"
```

연결이 맺어지면(`with` 블록 진입 성공) 포트가 열려 있다는 뜻이므로 성공으로 처리합니다.
 또한 `with` 문을 사용했으므로 연결은 블록을 벗어날 때 자동으로 닫힙니다.

- 연결 실패 처리하기


```
except Exception as e:
    return False, f"포트 닫힘: {e}"
```

연결이 거부되거나 타임아웃되면 예외가 발생하고, 이를 잡아 "포트 닫힘"으로 돌려줍니다.

실행해 보기 (REPL)

- 코드를 작성했으면 실제로 동작하는지 확인해 봅시다. GUI가 없어도 파이썬 대화형 셸(REPL)에서 클래스를 직접 만들어 실행할 수 있습니다.
- `checkers.py` 가 있는 폴더에서 터미널을 열고 파이썬 REPL을 실행합니다.

```
python
```

`>>>` 프롬프트가 나타나면 아래처럼 하나씩 입력합니다.

HTTP 점검 실행

- HTTP 점검

```
>>> from checkers import HttpChecker, IcmpChecker, TcpChecker
>>> HttpChecker("google.com").run()
(True, '상태코드 200')
```

`google.com` 이 정상 응답하면 위와 같이 `(True, '상태코드 200')` 이 출력됩니다. 존재하지 않는 주소를 넣으면 실패 튜플이 나옵니다.

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
Ctrl click to launch VS Code Native REPL
>>> from checkers import HttpChecker, TcpChecker, IcmpChecker
>>> HttpChecker("https://google.com").run()
(True, '상태코드 200')
>>> |
```

ICMP 점검 실행

- ICMP 점검

```
>>> IcmpChecker("8.8.8.8").run()
(True, '응답 성공')
```

8.8.8.8 은 구글의 공개 DNS 서버로 ping 테스트에 자주 쓰입니다. 사내망 등 ICMP가 차단된 환경에서는 **(False, '응답 없음')** 이 나올 수 있는데, 이는 코드 오류가 아니라 네트워크 정책 때문입니다.

TCP 점검 실행

- TCP 점검

```
>>> TcpChecker("naver.com", 80).run()
(True, '포트 열림')
```

80 은 웹 서버가 여는 표준 포트라 대개 열려 있습니다.

네트워크 점검 프로그램

앞선 'checker.py' 강좌에서 만든 점검 엔진 위에 최소한의 GUI를 씁니다.

- [최소한의 GUI 구성](#)
 - [목표](#)
 - [네트워크 점검 v0.1](#)
 - [코드 한 줄씩 살펴보기](#)
 - [check 메서드](#)
- [대상 등록 및 전체 점검](#)
 - [목표](#)
 - [네트워크 점검 v0.2](#)
 - [코드 한 줄씩 살펴보기](#)

최소한의 GUI 구성

목표

- 최소한의 GUI 최소한의 기능으로 HTTP, ICMP, TCP 점검만 합니다.

네트워크 점검 v0.1

- `checker.py` 와 같은 경로에 아래 코드 `main.py` 를 작성합니다.

```

import tkinter as tk
from tkinter import ttk, messagebox
from checker import HttpChecker, IcmpChecker, TcpChecker

TYPES = {"HTTP/HTTPS": HttpChecker, "ICMP": IcmpChecker, "TCP": TcpChecker}

class App:
    def __init__(self, root):
        root.geometry("560x180")

        top = ttk.Frame(root, padding=10)
        top.pack(fill="x")
        self.type_cb = ttk.Combobox(top, values=list(TYPES), state="readonly", width=12)
        self.type_cb.current(0)
        self.type_cb.pack(side="left")
        self.target_entry = ttk.Entry(top, width=25)
        self.target_entry.pack(side="left", padx=5)

        ttk.Label(top, text="포트:").pack(side="left")
        self.port_entry = ttk.Entry(top, width=8)
        self.port_entry.pack(side="left", padx=(2, 5))
        ttk.Button(top, text="점검", command=self.check).pack(side="left")

        self.result = ttk.Label(root, text="", padding=10, font=("", 12))
        self.result.pack()

    def check(self):
        target = self.target_entry.get().strip()
        if not target:
            messagebox.showwarning("입력 오류", "대상을 입력하세요.")
            return

        cls = TYPES[self.type_cb.get()]
        if cls is TcpChecker:
            port = self.port_entry.get().strip()
            if not port.isdigit():
                messagebox.showwarning("입력 오류", "TCP는 숫자 포트가 필요합니다.")
                return
            checker = cls(target, port)

```

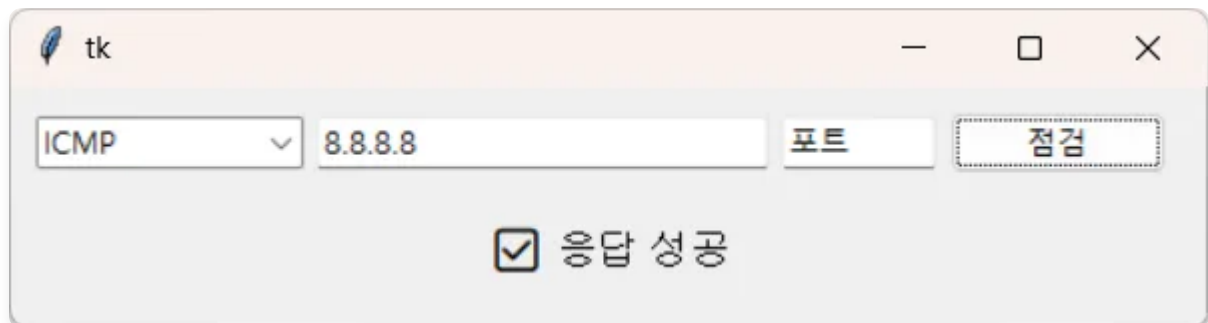
```

else:
    checker = cls(target)

    ok, msg = checker.run()
    self.result.config(text=f"{'✅' if ok else '❌'} {msg}")

if __name__ == "__main__":
    root = tk.Tk()
    App(root)
    root.mainloop()

```



코드 한 줄씩 살펴보기

- import와 재사용

```
from checker import HttpChecker, IcmpChecker, TcpChecker
```

점검 클래스 3개를 그대로 가져와 로직을 다시 짜지 않고 재사용합니다.

- TYPES 딕셔너리

```
TYPES = {"HTTP/HTTPS": HttpChecker, "ICMP": IcmpChecker, "TCP": TcpChecker}
```

- 화면 문자열("HTTP/HTTPS")과 실제 클래스(`HttpChecker`)를 이어 줍니다.
- 값에 괄호 없이 `HttpChecker` 로 저장 → 지금 실행하지 않고 클래스 자체를 보관, 버튼 클릭 시 객체 생성.

check 메서드

- `.get().strip()` 으로 대상 읽고, 비었으면 경고 후 `return` 합니다.
- `TYPES[선택값]` 으로 클래스를 꺼내고, `TcpChecker` 면 포트까지 넘겨 객체 생성합니다.
- `run()` 이 돌려준 `(ok, msg)` 를 라벨에 `config(text=...)` 로 표시. `ok` 에 따라  또는  를 표시합니다.

대상 등록 및 전체 점검

목표

- 점검 대상을 목록에 추가할 수 있도록 수정합니다.
- 목록에 추가된 대상을 전체 점검하고 결과를 확인할 수 있습니다.

네트워크 점검 v0.2

```

import tkinter as tk
from tkinter import ttk, messagebox
from checker import HttpChecker, IcmpChecker, TcpChecker

TYPES = {"HTTP/HTTPS": HttpChecker, "ICMP": IcmpChecker, "TCP": TcpChecker}

class App:
    def __init__(self, root):
        root.geometry("700x450")

        self.checkers = {}
        top = ttk.Frame(root, padding=10)
        top.pack(fill="x")
        self.type_cb = ttk.Combobox(top, values=list(TYPES), state="readonly", width=12)
        self.type_cb.current(0)
        self.type_cb.pack(side="left")
        self.target_entry = ttk.Entry(top, width=30)
        self.target_entry.pack(side="left", padx=5)
        ttk.Label(top, text="포트:").pack(side="left")
        self.port_entry = ttk.Entry(top, width=8)
        self.port_entry.pack(side="left", padx=(2, 5))
        ttk.Button(top, text="추가", command=self.add).pack(side="left")

        cols = ("type", "target", "result", "detail")
        self.tree = ttk.Treeview(root, columns=cols, show="headings")
        for c, t in zip(cols, ("유형", "대상", "결과", "상세")):
            self.tree.heading(c, text=t)
        self.tree.tag_configure("ok", foreground="green")
        self.tree.tag_configure("fail", foreground="red")
        self.tree.pack(fill="both", expand=True, padx=10)

        bottom = ttk.Frame(root, padding=10)
        bottom.pack(fill="x")
        ttk.Button(bottom, text="전체 점검", command=self.check_all).pack(side="left")
        ttk.Button(bottom, text="전체 삭제", command=self.clear).pack(side="left", padx=5)

    def add(self):
        ctype = self.type_cb.get()
        cls = TYPES[ctype]

```

```

target = self.target_entry.get().strip()
port = self.port_entry.get().strip()
if not target:
    messagebox.showwarning("입력 오류", "대상을 입력하세요.")
    return

if cls is TcpChecker:
    if not port.isdigit():
        messagebox.showwarning("입력 오류", "TCP는 숫자 포트가 필요합니다.")
        return
    checker = cls(target, port)
    shown = f"{target}:{port}"
else:
    checker = cls(target)
    shown = target

iid = self.tree.insert("", tk.END, values=(ctype, shown, "-", ""))
self.checkers[iid] = checker
self.target_entry.delete(0, tk.END)

def check_all(self):
    for iid, checker in self.checkers.items():
        try:
            ok, detail = checker.run()
        except Exception as e:
            self.tree.set(iid, "result", "오류")
            self.tree.set(iid, "detail", str(e))
            self.tree.item(iid, tags=("fail",))
            continue

        self.tree.set(iid, "result", "정상" if ok else "실패")
        self.tree.set(iid, "detail", detail)
        self.tree.item(iid, tags=("ok" if ok else "fail",))

def clear(self):
    for iid in self.tree.get_children():
        self.tree.delete(iid)
    self.checkers.clear()

if __name__ == "__main__":

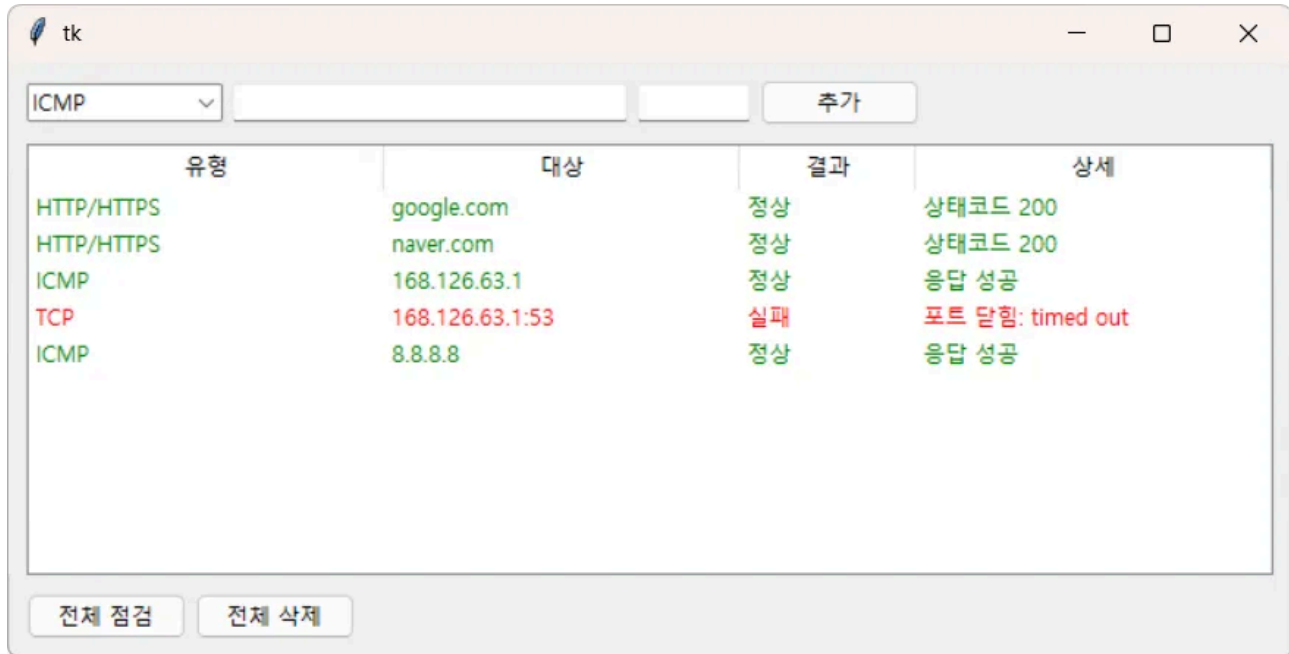
```



```

root = tk.Tk()
App(root)
root.mainloop()

```



코드 한 줄씩 살펴보기

- 대상 추가 (`add`)

```

def add(self):
    ctype = self.type_cb.get()
    cls = TYPES[ctype]
    target = self.target_entry.get().strip()
    port = self.port_entry.get().strip()

```

드롭다운에서 고른 유형, 대상 입력값, 포트 입력값을 각각 읽어 옵니다. `strip()` 으로 앞뒤 공백을 제거합니다.

- 전체 점검 (`check_all`)

```
def check_all(self):
    for iid, checker in self.checkers.items():
```

- 저장된 모든 점검 객체를 하나씩 꺼내 `run()` 으로 점검을 실행하고, 성공 여부 `ok` 와 상세 메시지 `detail` 을 받습니다.
- 해당 행의 "결과" 열에 정상/실패를, "상세" 열에 메시지를 채워 넣습니다.

- 전체 삭제 (`clear`)

```
def clear(self):
    for iid in self.tree.get_children():
        self.tree.delete(iid)
    self.checkers.clear()
```

`get_children()` 로 표의 모든 행을 가져와 하나씩 삭제하고, 점검 객체를 담아 둔 사전도 비웁니다.

네트워크 점검 스레드 적용

앞에서 만든 네트워크 점검 앱은 점검 대상이 여러 개일 때 '전체 점검'을 누르면 점검이 끝날 때까지 창이 멈춘 것처럼 반응하지 않습니다.

이번에는 이런 현상이 발생하는 이유를 살펴보고, 스레드(thread)와 큐(queue)를 이용해 네트워크 점검 중에도 창이 정상적으로 반응하도록 수정해 보겠습니다.

- [원인과 해결](#)
 - [창이 멈추는 원인](#)
 - [해결 방법](#)
- [스레드와 큐를 적용](#)
 - [전체 코드](#)
 - [코드 한 줄씩 살펴보기](#)

원인과 해결

창이 멈추는 원인

- 메인 스레드 하나에서 '네트워크 점검'과 화면을 갱신하는 작업을 모두 혼자 처리하기 때문입니다.

해결 방법

- 시간이 오래 걸리는 '네트워크 점검'은 백그라운드 스레드에게 맡기고, 메인 스레드는 화면 처리에만 집중하게 합니다.
 - 메인 스레드: 화면 갱신만 담당.
 - 백그라운드 스레드: 네트워크 점검(`checker.run()`)만 담당, 점검 결과를 큐에 저장.
- 메인 스레드와 백그라운드 스레드 사이에 큐(queue)를 다리처럼 놓습니다.
 - 백그라운드 스레드는 점검 결과를 큐에 넣기만 함.
 - 메인 스레드는 큐에서 결과를 꺼내서 화면에 반영.

스레드와 큐를 적용

전체 코드

- 스레드와 큐를 적용한 코드입니다.

```

import tkinter as tk
from tkinter import ttk, messagebox
import threading
import queue
from checker import HttpChecker, IcmpChecker, TcpChecker

TYPES = {"HTTP/HTTPS": HttpChecker, "ICMP": IcmpChecker, "TCP": TcpChecker}

class App:
    def __init__(self, root):
        self.root = root
        root.geometry("700x450")

        self.checkers = {}
        self.result_queue = queue.Queue()

        top = ttk.Frame(root, padding=10)
        top.pack(fill="x")
        self.type_cb = ttk.Combobox(top, values=list(TYPES), state="readonly", width=12)
        self.type_cb.current(0)
        self.type_cb.pack(side="left")
        self.target_entry = ttk.Entry(top, width=30)
        self.target_entry.pack(side="left", padx=5)
        ttk.Label(top, text="포트:").pack(side="left")
        self.port_entry = ttk.Entry(top, width=8)
        self.port_entry.pack(side="left", padx=(2, 5))
        ttk.Button(top, text="추가", command=self.add).pack(side="left")

        cols = ("type", "target", "result", "detail")
        self.tree = ttk.Treeview(root, columns=cols, show="headings")
        for c, t in zip(cols, ("유형", "대상", "결과", "상세")):
            self.tree.heading(c, text=t)
        self.tree.tag_configure("ok", foreground="green")
        self.tree.tag_configure("fail", foreground="red")
        self.tree.pack(fill="both", expand=True, padx=10)

        bottom = ttk.Frame(root, padding=10)
        bottom.pack(fill="x")
        self.check_btn = ttk.Button(bottom, text="전체 점검", command=self.check_all)

```

```

self.check_btn.pack(side="left")
ttk.Button(bottom, text="전체 삭제", command=self.clear).pack(side="left", padx=5)

self.root.after(100, self.process_queue)

def add(self):
    ctype = self.type_cb.get()
    cls = TYPES[ctype]
    target = self.target_entry.get().strip()
    port = self.port_entry.get().strip()
    if not target:
        messagebox.showwarning("입력 오류", "대상을 입력하세요.")
        return

    if cls is TcpChecker:
        if not port.isdigit():
            messagebox.showwarning("입력 오류", "TCP는 숫자 포트가 필요합니다.")
            return
        checker = cls(target, port)
        shown = f"{target}:{port}"
    else:
        checker = cls(target)
        shown = target

    iid = self.tree.insert("", tk.END, values=(ctype, shown, "-", ""))
    self.checkers[iid] = checker
    self.target_entry.delete(0, tk.END)

def check_all(self):
    if not self.checkers:
        return

    self.check_btn.config(state="disabled")
    for iid in self.checkers:
        self.tree.set(iid, "result", "점검 중...")
        self.tree.set(iid, "detail", "")
        self.tree.item(iid, tags=())

    targets = list(self.checkers.items())
    thread = threading.Thread(target=self.run_checks, args=(targets,), daemon=True)
    thread.start()

```

```

def run_checks(self, targets):
    for iid, checker in targets:
        try:
            ok, detail = checker.run()
            self.result_queue.put((iid, "정상" if ok else "실패", detail,
                                   "ok" if ok else "fail"))
        except Exception as e:
            self.result_queue.put((iid, "오류", str(e), "fail"))
    self.result_queue.put(("__done__", None, None, None))

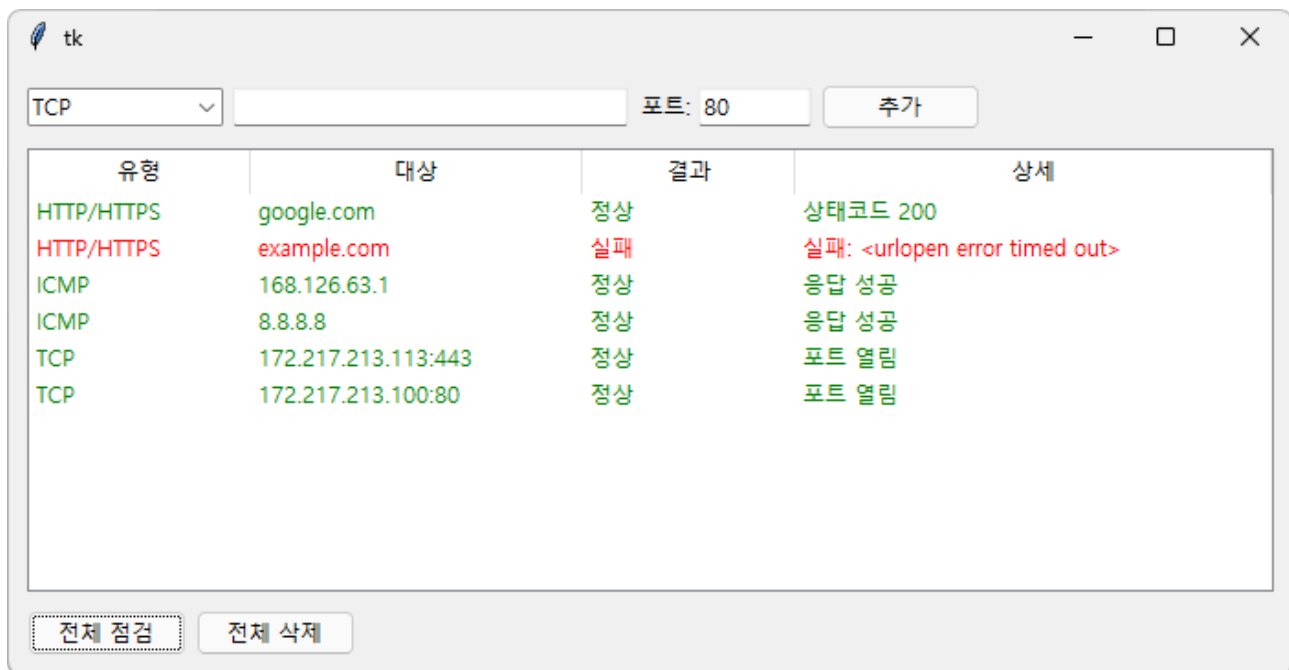
def process_queue(self):
    try:
        while True:
            iid, result, detail, tag = self.result_queue.get_nowait()
            if iid == "__done__":
                self.check_btn.config(state="normal")
                continue

            if self.tree.exists(iid):
                self.tree.set(iid, "result", result)
                self.tree.set(iid, "detail", detail)
                self.tree.item(iid, tags=(tag,))
    except queue.Empty:
        pass
    finally:
        self.root.after(100, self.process_queue)

def clear(self):
    for iid in self.tree.get_children():
        self.tree.delete(iid)
    self.checkers.clear()

if __name__ == "__main__":
    root = tk.Tk()
    App(root)
    root.mainloop()

```



코드 한 줄씩 살펴보기

- 스레드와 큐 준비

```
import threading
import queue
```

```
self.result_queue = queue.Queue()
```

- `queue` 는 스레드 사이에 데이터를 안전하게 주고받기 위한 표준 라이브러리입니다.
- 백그라운드 스레드는 큐에 결과를 넣기만 하고, 메인 스레드는 꺼내기만 합니다. 위젯을 직접 건드리지 않고 큐를 사이에 두는 것이 핵심입니다.

- 큐를 주기적으로 확인하도록 예약

```
self.root.after(100, self.process_queue)
```


- `after(100, 함수)` 는 "100밀리초 뒤에 이 함수를 메인 스레드에서 실행하라"는 예약입니다.
- 예약된 콜백은 메인 스레드에서 실행되므로, 그 안에서 위젯을 조작하는 것이 안전합니다.

- 점검 시작 (`check_all`)

```
def check_all(self):
    if not self.checkers:
        return

    self.check_btn.config(state="disabled")
    for iid in self.checkers:
        self.tree.set(iid, "result", "점검 중...")
        self.tree.set(iid, "detail", "")
        self.tree.item(iid, tags=())

    targets = list(self.checkers.items())
    thread = threading.Thread(target=self.run_checks, args=(targets,), daemon=True)
    thread.start()
```

- 버튼을 비활성화해 점검 도중 중복 실행을 막고, 각 행에 "점검 중..."을 표시합니다.
- `targets` 로 대상 목록을 복사해, 점검 도중 '전체 삭제'로 원본이 바뀌어도 스레드는 영향받지 않습니다.
- `daemon=True` 는 창이 종료되면 스레드도 함께 종료하라는 뜻입니다.
- `thread.start()` 순간 점검은 백그라운드로 넘어가고, 메인 스레드는 곧바로 화면 처리로 돌아가므로 앱이 멈추지 않습니다.

- 백그라운드에서 점검 실행 (`run_checks`)

```
def run_checks(self, targets):
    for iid, checker in targets:
        try:
            ok, detail = checker.run()
            self.result_queue.put((iid, "정상" if ok else "실패", detail,
                                   "ok" if ok else "fail"))
        except Exception as e:
            self.result_queue.put((iid, "오류", str(e), "fail"))
    self.result_queue.put(("__done__", None, None, None))
```

- 이 함수는 백그라운드 스레드에서 실행되며, 위젯을 전혀 건드리지 않고 결과를 `result_queue.put(...)` 으로 큐에 넣기만 합니다.
- 한 대상에서 예외가 나도 `try/except` 로 잡아 "오류"로 넣고 다음 대상으로 넘어갑니다.
- 모두 끝나면 `("__done__", ...)` 을 넣어 점검 완료를 알립니다.

- 메인 스레드에서 화면 갱신 (`process_queue`)

```
def process_queue(self):
    try:
        while True:
            iid, result, detail, tag = self.result_queue.get_nowait()
            if iid == "__done__":
                self.check_btn.config(state="normal")
                continue
            if self.tree.exists(iid):
                self.tree.set(iid, "result", result)
                self.tree.set(iid, "detail", detail)
                self.tree.item(iid, tags=(tag,))
    except queue.Empty:
        pass
    finally:
        self.root.after(100, self.process_queue)
```

- `after` 로 예약되어 메인 스레드에서 100밀리초마다 실행되므로 위젯 조작이 안전합니다.
- `get_nowait()` 는 큐가 비면 기다리지 않고 바로 예외를 내므로, `while`로 쌓인 결과를 한번에 처리하고 비면 조용히 빠져나옵니다. 기다리지 않는 방식을 쓰는 이유는 여기서 멈추면 화면이 다시 얼어붙기 때문입니다.
- `exists(iid)` 로 행이 아직 살아 있는지 확인한 뒤에만 갱신합니다. '전체 삭제'로 이미 지워졌을 수 있기 때문입니다.
- `finally` 에서 다시 자기 자신을 예약해, 앱이 켜져 있는 동안 계속 큐를 확인하는 순환을 이룹니다.